UNIVERSIDAD AUTÓNOMA “GABRIEL RENÉ MORENO”

FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

PRIMER PARCIAL

“SOFTWARE DE ASISTENCIA”

MATERIA	:	INGENIERIA DE SOFTWARE
DOCENTE	:	ING. JOSUE OBED VEIZAGA GONZALES
FECHA	:	28/09/2025
ESTUDIANTE	:	JHOEL DEBRAY QUISPE TICONA
REGISTRO	:	215162617

SANTA CRUZ – BOLIVIA

1 Tabla de contenido

2 REQUISITOS 1

2.1 Requisitos funcionales: 1

2.1.1 Identificar Casos de uso 1

2.1.2 Detalles de caso de uso 1

2.2 Requisitos no funcionales 13

2.2.1 Rendimiento 13

2.2.2 Usabilidad 13

2.2.3 Escalabilidad 13

2.2.4 Compatibilidad 14

2.2.5 Mantenimiento 14

2.2.6 Disponibilidad 14

2.2.7 Portabilidad 14

3 ANALISIS 14

3.1 Identificación de los módulos 14

3.1.1 Modulo Administración 14

3.1.2 Modulo Asistencia 15

3.2 Vista de módulos 15

3.2.1 Modulo Administración 15

3.2.2 Modulo Asistencia 16

4 DISEÑO 16

4.1 Diseño de la Arquitectura Lógica 16

4.1.1 Administración 16

4.1.2 Asistencia 16

4.2 Diseño de la base de datos 17

4.2.1 Diseño conceptual 17

4.2.2 Diseño Lógico 17

4.2.3 Diseño Físico 19
4.3 Diseño de Detalle Procedimental 20
4.3.1 Diagrama de clases dinamico 20
4.3.1 Diagrama de secuencia 24
4.4 Diseño de la interfaz 30
4.4.1 CU01: Gestionar Estudiantes 30
4.4.2 CU02: Gestionar Materias 30
4.4.3 CU03: Gestionar Grupo 30
4.4.4 CU04: Gestionar Horario 31
4.4.5 CU05: Registrar Asistencia QR 31
4.4.6 CU06: Gestionar Aula 31
5 Bibliografia 31

2 REQUISITOS

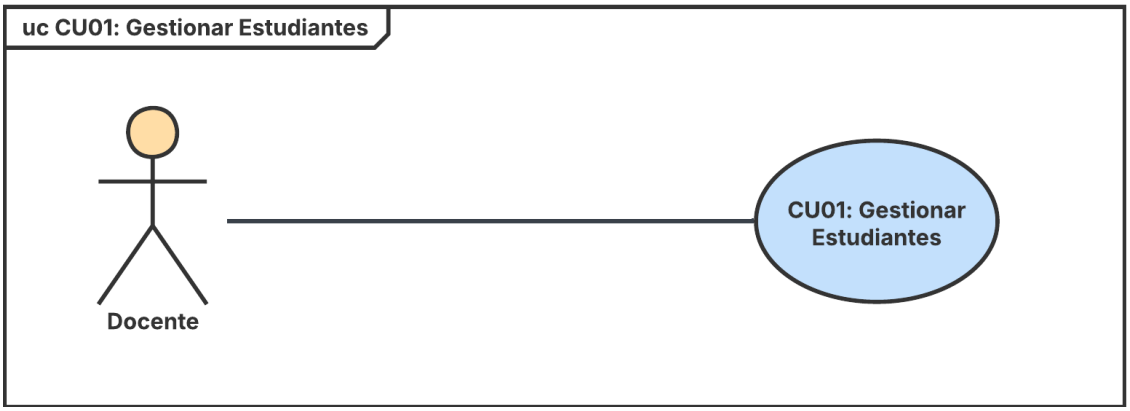
2.1 Requisitos funcionales:

2.1.1 Identificar Casos de uso

- CU01: Gestionar Estudiantes
- CU02: Gestionar Materias
- CU03: Gestionar Grupos
- CU04: Gestionar Horario
- CU05: Registrar Asistencia QR
- CU06: Gestionar Aula

2.1.2 Detalles de caso de uso

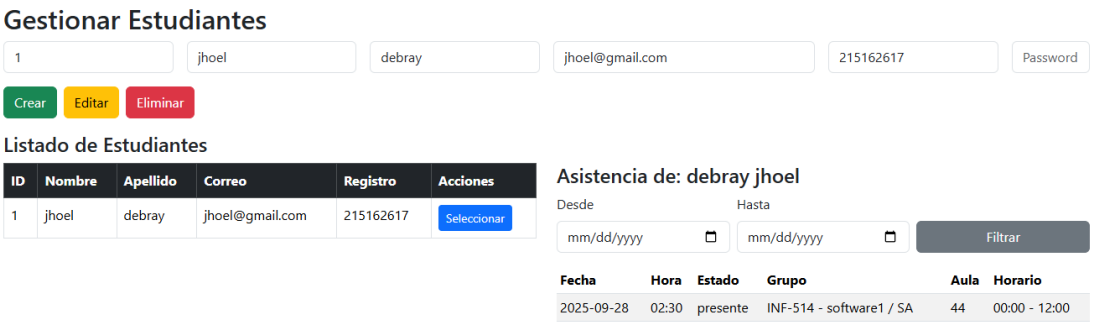
2.1.2.1 CU01: Gestionar Estudiantes



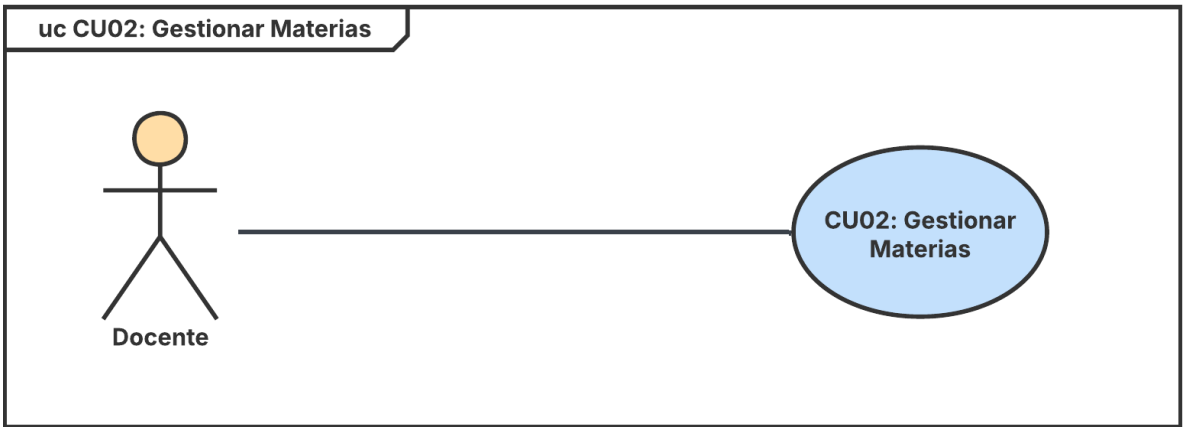
Nombre de caso de uso	CU01: Gestionar Estudiantes
Propósito	Gestionar los registros de Estudiantes
Descripción	Permite registrar, modificar y eliminar Estudiantes registrados como también ver la lista de asistencia de cada uno
Actores	Docente
Actor iniciador	Docente
Pre condición	Ninguna
Flujo principal	**Registrar Estudiante** 1. El Docente selecciona **Registrar estudiante**. 2. El sistema muestra formulario: *nombre, apellido, correo_electrónico, contraseña*. 3. El Docente completa los campos obligatorios. 4. El sistema valida: campos, formato. 5. El sistema crea el Usuario. 6. El sistema actualiza y muestra la lista de estudiantes del Docente. **Modificar Estudiante** 1. El Docente selecciona un estudiante. 2. El sistema muestra los datos actuales. 3. El Docente edita nombre, apellido, correo. 4. El sistema valida y guarda cambios (actualiza **Usuario**). 5. Se actualiza la lista. **Eliminar Estudiante** 1. El Docente selecciona Eliminar Estudiante. 2. El sistema solicita confirmación. 3. El sistema aplica la acción y actualiza la lista. **Listar Estudiante** 1. El Docente desde el menú principal selecciona Estudiante. 2. El sistema muestra una presentación de los estudiantes completa **Listar Asistencia** 3. El Docente selecciona el Estudiante. 4. El sistema muestra una lista de asistencia del estudiante seleccionado
Postcondicion	1. El estudiante queda registrado, modificado o eliminado según corresponda. 2. Las listas de estudiantes del Docente reflejan

	los cambios.
Excepción	1. Correo ya existente. 2. Formato de correo inválido. 3. Intento de guardar con campos obligatorios vacíos.

2.1.2.1.1 Prototipos



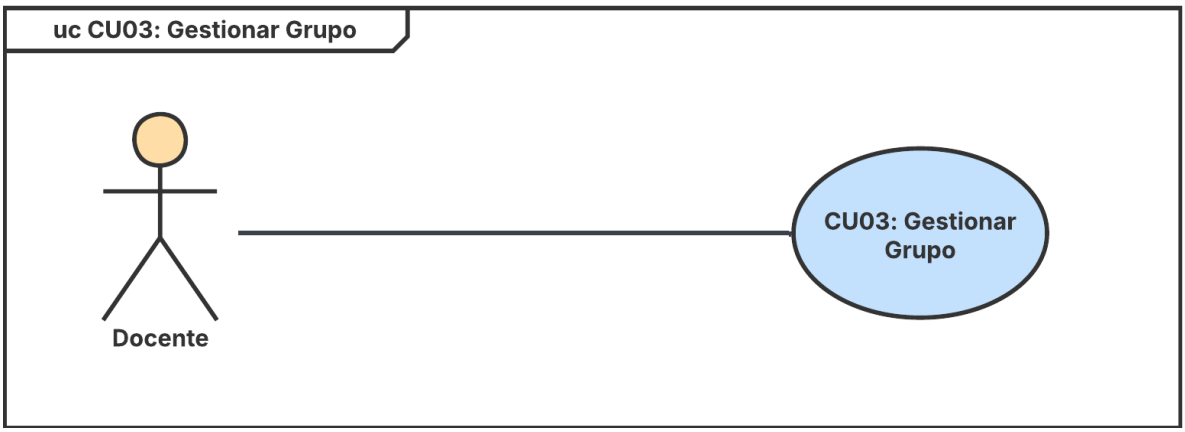
2.1.2.2 CU02: Gestionar Materias



Nombre de caso de uso	CU02: Gestionar Materias
Propósito	Permitir al **Docente** crear, modificar y eliminar materias del sistema.
Descripción	Permite al docente Gestionar sus **Materias** sigla, nombre materia, nivel.
Actores	Docente
Actor iniciador	Docente
Pre condición	Ninguna.
Flujo principal	**Registrar Materia** 1. El Docente selecciona **Registrar materia**. 2. El sistema muestra formulario con: **sigla**, **nombre materia**, **nivel**. 3. El Docente completa los campos obligatorios. 4. El sistema valida: Campos obligatorios llenos. 5. El sistema guarda la **Materia** y muestra la lista actualizada. **Modificar Materia** 1. El Docente elige una materia de la lista. 2. El sistema muestra sus datos actuales. 3. El Docente edita sigla, nombre materia y/o nivel. 4. El sistema valida. 5. El sistema guarda y actualiza la lista. 6. Los **Grupos** asociados permanecen vinculados a la materia editada. **Eliminar Materia** 1. El Docente selecciona **Eliminar** sobre una materia. 2. El sistema solicita confirmación. 3. El sistema verifica relaciones 4. **Si no existen relaciones**, procede con la eliminación. 5. El sistema aplica la acción y actualiza la lista. **Listar Materia** 5. El Docente desde el menú principal selecciona Materia. 6. El sistema muestra una presentación de las materias completa
Postcondicion	1. La materia queda creada, modificada, eliminada o desactivada correctamente. 2. La lista de materias refleja los cambios.
Excepción	1. sigla duplicada. 2. Intento de guardar con campos obligatorios vacíos.

2.1.2.2.1 Prototipos

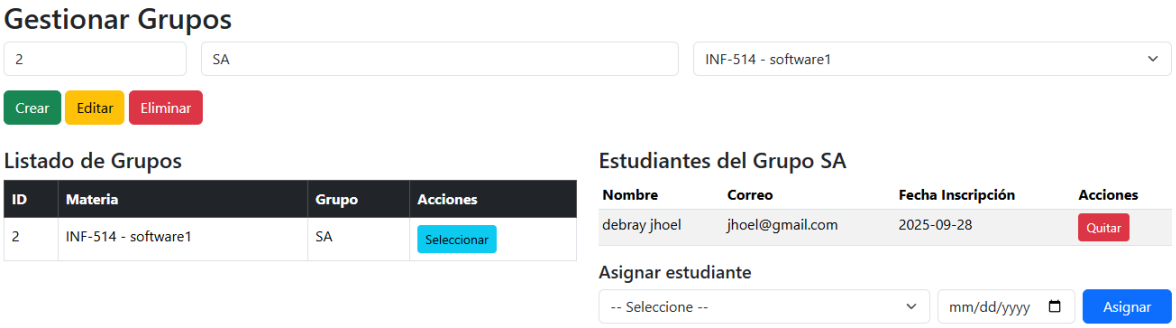
2.1.2.3 CU03: Gestionar Grupo



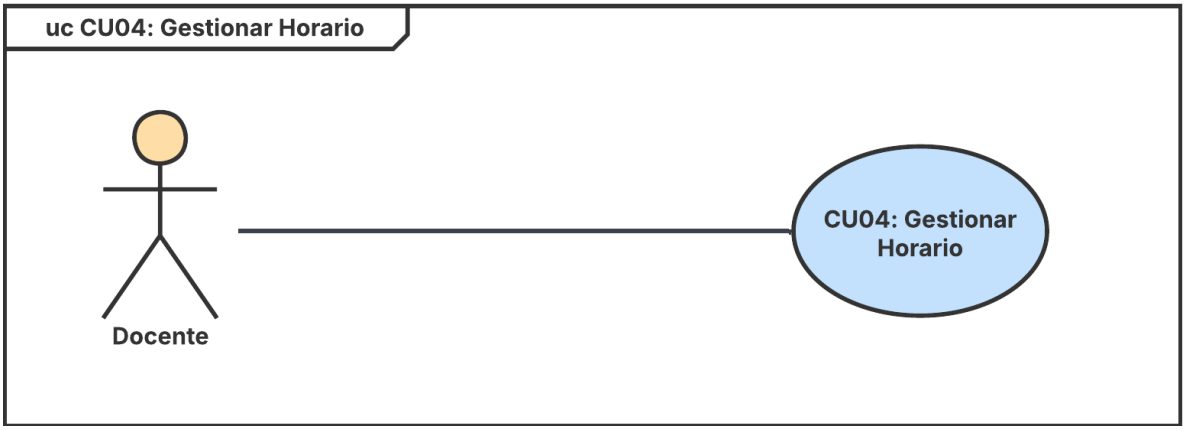
Nombre de caso de uso	CU03: Gestionar Grupo
Propósito	Permitir al **Docente** crear, modificar y eliminar **grupos** en el sistema como también asignar o quitar estudiante de grupos.
Descripción	El Docente gestiona sus **Grupos** pudiendo crear, editar, eliminar, listar además de asignar y quitar estudiantes del grupo.
Actores	Docente
Actor iniciador	Docente
Pre condición	1. Debe existir al menos una **Materia** registrada para asociar el grupo.
Flujo principal	**Registrar grupo** 1. El Docente selecciona **Registrar grupo**. 2. El sistema muestra formulario con: **nombre** *(obligatorio)*, **materia**. 3. El Docente completa los campos. 4. El sistema **valida**: campos obligatorios, nombre no vacío. 5. El sistema **guarda** el Grupo y muestra la **lista actualizada**. **Modificar grupo** 1. El Docente elige un grupo de la lista. 2. El sistema muestra sus datos actuales. 3. El Docente edita **nombre** y/o **materia** (si aplica). 4. El sistema valida (campos, duplicados en la misma materia). 5. El sistema **actualiza** el Grupo y muestra la lista. **Eliminar grupo** 1. El Docente selecciona **Eliminar** sobre un grupo. 2. El sistema solicita **confirmación**. 3. El sistema **verifica relaciones** 4. El sistema **elimina** **Listar Grupo** 1. El Docente desde el menú principal selecciona Materia. 2. El sistema muestra una presentación de los grupos completa **Asignar Estudiante a grupo** 1. Selecciona un grupo. 2. En el formulario de asignación elegir un estudiante del select y la fecha de unión 3. Dar click en “asignar” **Quitar Estudiante a grupo** 1. Selecciona un grupo.

	2. Visualizar la tabla de estudiantes asignados 3. Dar click en “quitar”
Postcondicion	1. El grupo queda **creado, modificado, eliminado** correctamente. 2. La **lista de grupos** refleja los cambios.
Excepción	1. **Nombre duplicado** en la misma materia. 2. Intento de guardar con **campos obligatorios** vacíos.

2.1.2.3.1 Prototipos



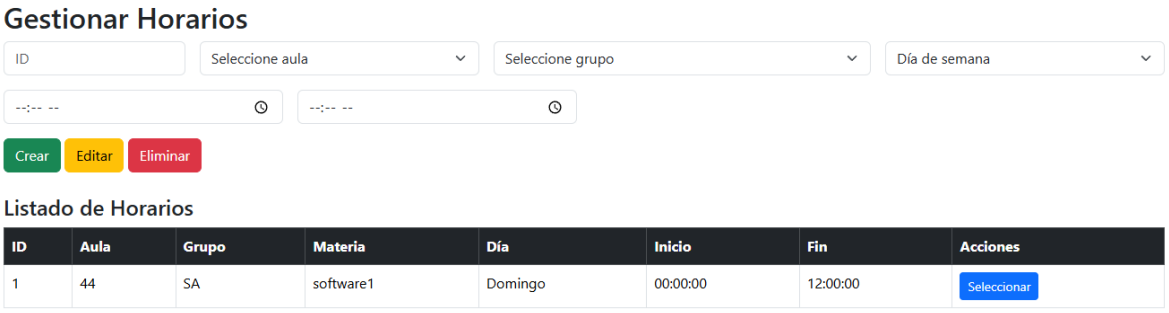
2.1.2.4 CU04: Gestionar Horario



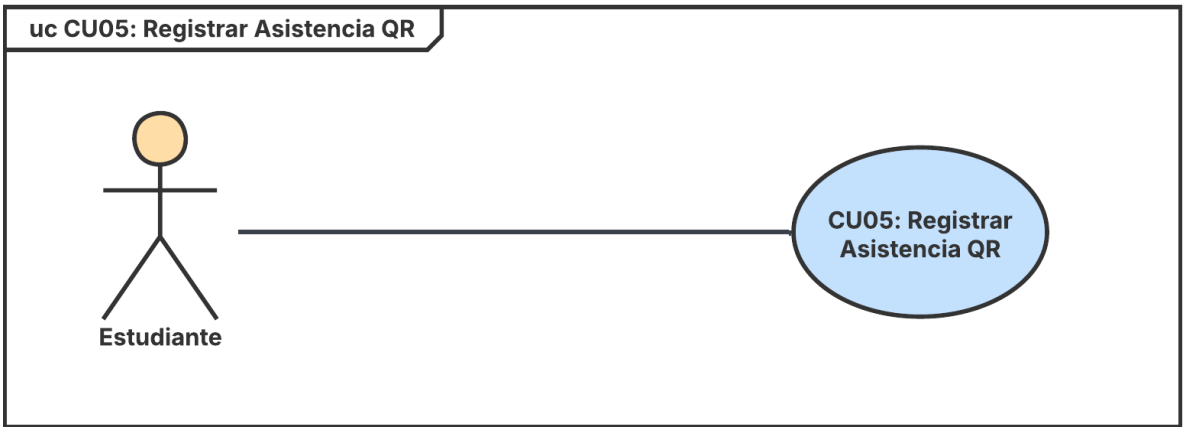
Nombre de caso de uso	CU03: Gestionar Horario
Propósito	Permitir al **Docente** crear, modificar y eliminar **horarios** de clase asociados a un grupo y a un aula.
Descripción	El docente gestiona los horarios indicando grupo, aula, día de la semana y horas de inicio/fin. Esto define cuándo y dónde se dictan las materias y permite posteriormente controlar la asistencia.
Actores	Docente
Actor iniciador	Docente
Pre condición	1. Debe existir al menos un **Grupo** y un **Aula** registrados.
Flujo principal	**Registrar Horario** 1. El Docente selecciona **Registrar horario**. 2. El sistema muestra formulario con: **grupo**, **aula**, **día de semana**, **hora inicio**, **hora fin**. 3. El Docente completa los campos. 4. El sistema valida: Campos obligatorios llenos. 5. **Rangos horarios válidos** (hora inicio < hora fin). 6. El sistema guarda el **Horario** y muestra la lista actualizada. **Modificar Horario** 1. El Docente selecciona un horario existente de la lista. 2. El sistema muestra los datos actuales. 3. El Docente edita: aula, día, hora inicio y/o hora fin. 4. El sistema valida los cambios (campos obligatorios, rangos válidos, no solapamiento). 5. El sistema guarda y muestra la lista de horarios actualizada. **Eliminar horario** 1. El Docente selecciona **Eliminar** sobre un horario. 2. El sistema solicita confirmación. 3. El sistema verifica relaciones: 4. Se procede con la eliminación. 5. El sistema aplica la acción y actualiza la lista. **Listar Grupo** 1. El Docente desde el menú principal selecciona Horario. 2. El sistema muestra una presentación de los Horarios completa

Postcondicion	1. El horario queda creado, modificado, eliminado correctamente. 2. La lista de horarios refleja los cambios. 3. Se mantiene la integridad: no hay choques de horarios ni registros huérfanos de asistencia.
Excepción	1. Campos obligatorios vacíos. 2. Hora inicio ≥ hora fin.

2.1.2.4.1 Prototipos

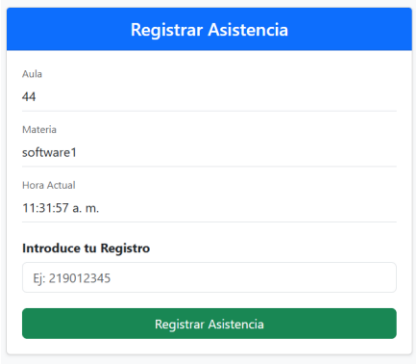


2.1.2.5 CU5: Registrar Asistencia QR

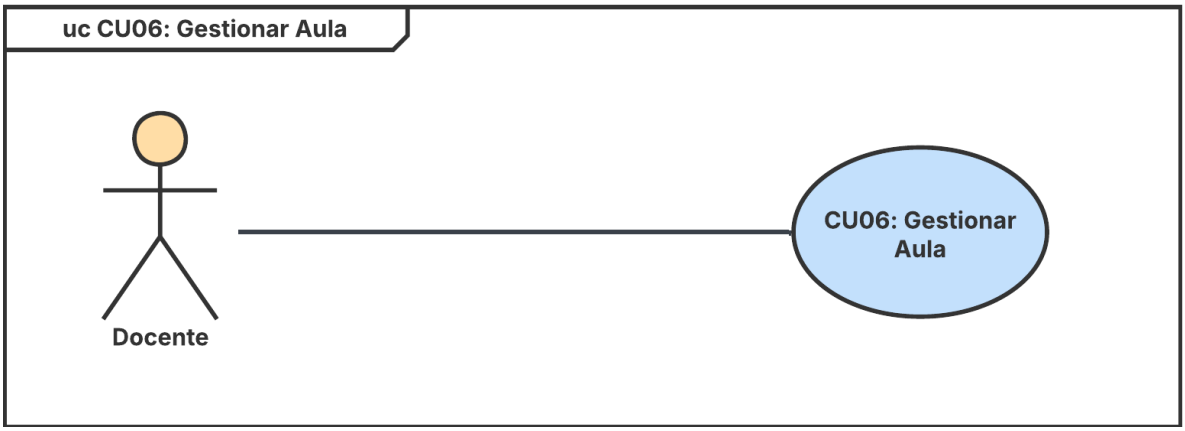


Nombre de caso de uso	CU5. Registrar asistencia QR
Propósito	Permitir al **Estudiante** registrar su asistencia mediante el escaneo de un código QR y relleno de un fomulario.
Descripción	El estudiante utiliza su dispositivo para escanear el **QR** asociado al aula/horario en curso. El sistema valida el código, la hora y la relación estudiante-grupo y registra la Asistencia.
Actores	Estudiante
Actor iniciador	Estudiante
Pre condición	1. El estudiante debe estar inscrito en el grupo (Inscripcion).
Flujo principal	**Registrar ministerio** 1. El estudiante escanea el código qr que previamente a sido generado por el docente 2. El estudiante verifica la información y coloca su numero de registro 3. El estudiante selecciona la opción **Registrar asistencia**. 4. El sistema valida que corresponda a un **aula y horario** vigentes, que la hora de registro esté dentro del rango válido (hora_inicio y hora_fin), que el estudiante esté inscrito en el grupo correspondiente. 5. Si las validaciones son correctas, el sistema crea un registro en Asistencia con: • Id de estudiante • Id de horario • Fecha y hora • Estado de asistencia = presente 6. El sistema confirma la asistencia registrada al estudiante.
Postcondicion	1. La asistencia queda registrada en la tabla Asistencia. 2. El estudiante recibe confirmación (ej. mensaje en pantalla). 3. La asistencia puede ser consultada posteriormente por el docente.
Excepción	1. **Estudiante no inscrito**: El estudiante no se encuentra inscrito 2. **Asistencia ya registrada**: el estudiante intenta registrar más de una vez en la misma sesión.

2.1.2.5.1 Prototipos



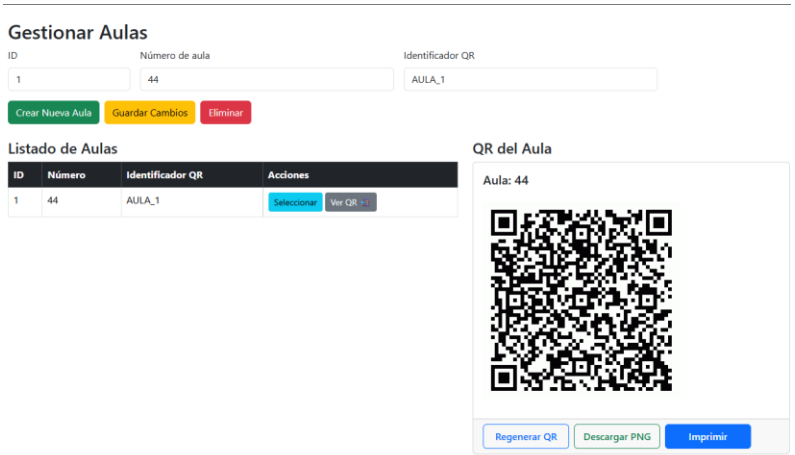
2.1.2.6 CU6: Gestionar Aula



Nombre de caso de uso	CU6. Gestionar Aula
Propósito	Permitir al **Docente** crear, actualizar y eliminar **aulas**, así como **generar** el código **QR** que se utilizará para el registro de asistencia.
Descripción	El Docente administra las aulas del sistema. Cada aula posee un **código** único y un **QR** que puede generarse.
Actores	Docente
Actor iniciador	Docente
Pre condición	1. Ninguna.
Flujo principal	**Registrar Aula** 1. El Docente selecciona **Registrar aula**. 2. El sistema muestra formulario con: **código** (obligatorio, único) y **opción para generar QR** (opcional en el alta). 3. El Docente completa los campos. 4. El sistema **valida**: código no vacío y **no duplicado**. 5. El sistema **guarda** el Aula **genera** y asocia qr_code. 6. El sistema muestra la **lista de aulas** actualizada. **Modificar Aula** 1. El Docente elige un aula de la lista. 2. El sistema muestra los datos actuales. 3. El Docente edita el **código** 4. El sistema valida (código no vacío, **no duplicado**). 5. El sistema **actualiza** el aula. 6. El sistema muestra la lista actualizada. 7. Los **Horarios** asociados **se mantienen** vinculados al aula. **Eliminar Aula** 1. El Docente selecciona **Eliminar** sobre un aula. 2. El sistema solicita **confirmación**. 3. El sistema **elimina** el aula y actualiza la lista. **Generar QR de sesión** 1. El Docente hace clic en **Generar QR** (o **Regenerar QR**). 2. El sistema genera el código qr 3. El sistema **asocia** el código qr al aula y lo **muestra** para que sea compartido.
Postcondicion	1. El aula queda **creada, modificada, eliminada/desactivada** correctamente. 2. La **lista de aulas** refleja los cambios.

	3. Si se generó QR, el aula queda con un código qr **vigente** y listo para ser utilizado en registro de asistencia.
Excepción	1. **Código duplicado** de aula. 2. Intento de guardar con **campos obligatorios** vacíos.

2.1.2.6.1 Prototipos



2.2 Requisitos no funcionales

2.2.1 Rendimiento

- El sistema deberá permitir cargar cada caso de uso (Aula, Cursos, Estudiantes, etc.) en menos de 3 segundos con una conexión estándar a internet.
- La inserción, edición o eliminación de datos debe reflejarse de forma inmediata en la interfaz del usuario.

2.2.2 Usabilidad

- La interfaz del sistema debe ser clara, intuitiva y fácil de usar para usuarios no técnicos.

2.2.3 Escalabilidad

- El sistema debe permitir aumentar la capacidad de usuarios y módulos sin necesidad de rediseñar la arquitectura.
- Deberá soportar al menos 500 registros por entidad sin degradación del rendimiento.

2.2.4 Compatibilidad

- El sistema deberá funcionar correctamente en navegadores modernos como Chrome, Firefox y Edge (últimas 2 versiones).
- Compatible con sistemas operativos Windows, Linux y macOS vía navegador.

2.2.5 Mantenimiento

- El código deberá estar estructurado siguiendo el patrón arquitectónico de 3 capas para facilitar su mantenimiento.
- Los archivos deberán estar organizados por módulos funcionales y separados por responsabilidades (presentación, negocio, datos).

2.2.6 Disponibilidad

- El sistema estará disponible para uso al menos el 95% del tiempo durante los días hábiles.
- El software deberá poder ser accedido 24/7 desde cualquier navegador web compatible.

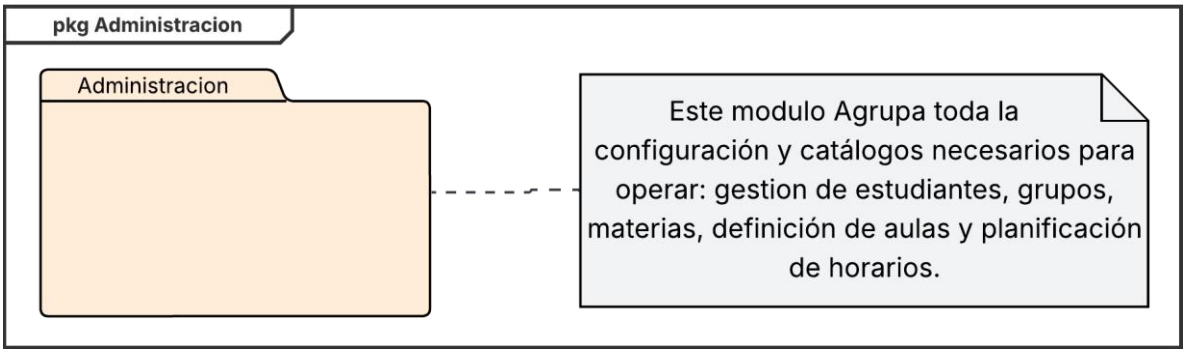
2.2.7 Portabilidad

El sistema podrá ser desplegado localmente en servidores con Apache y PHP, o en servicios en la nube como AWS, Heroku o similar.

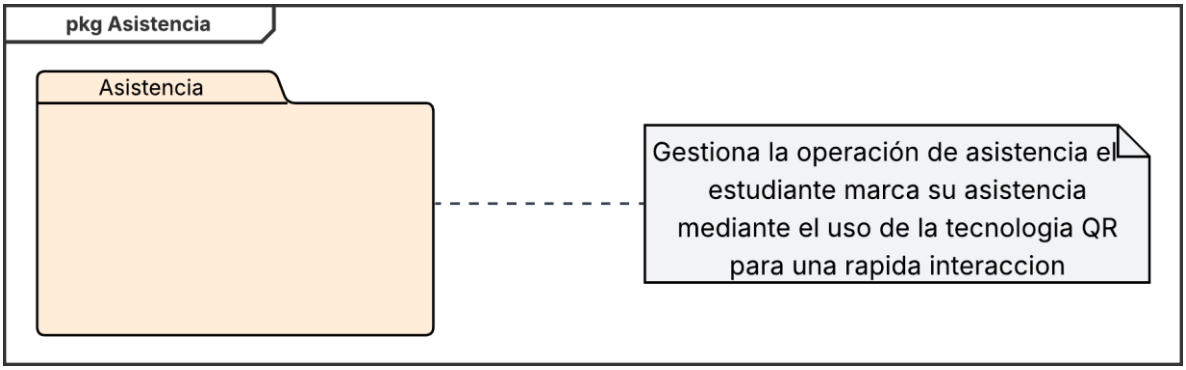
3 ANALISIS

3.1 Identificación de los módulos

3.1.1 Modulo Administración

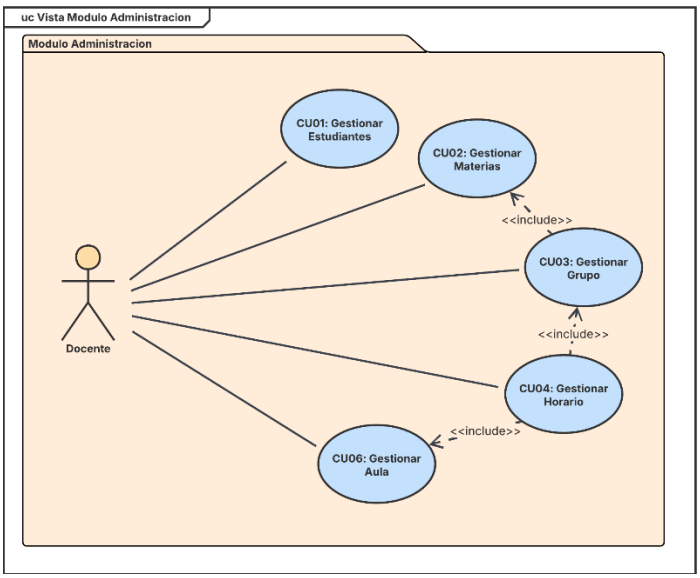


3.1.2 Modulo Asistencia

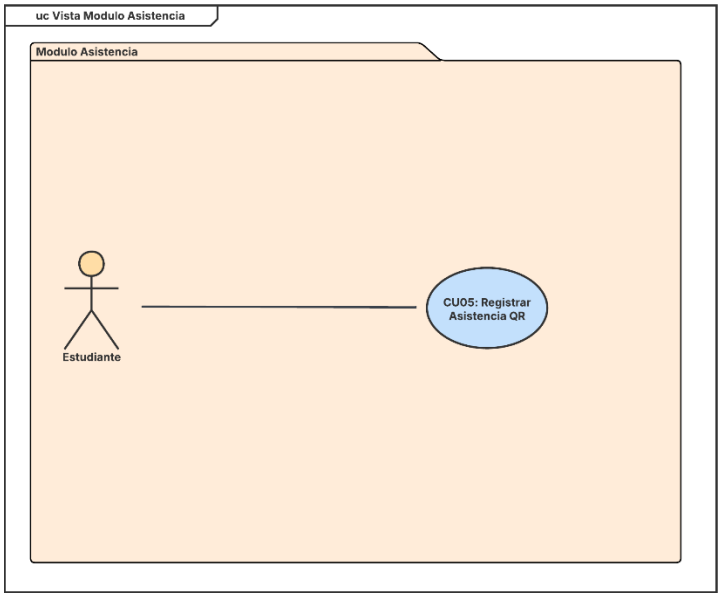


3.2 Vista de módulos

3.2.1 Modulo Administración



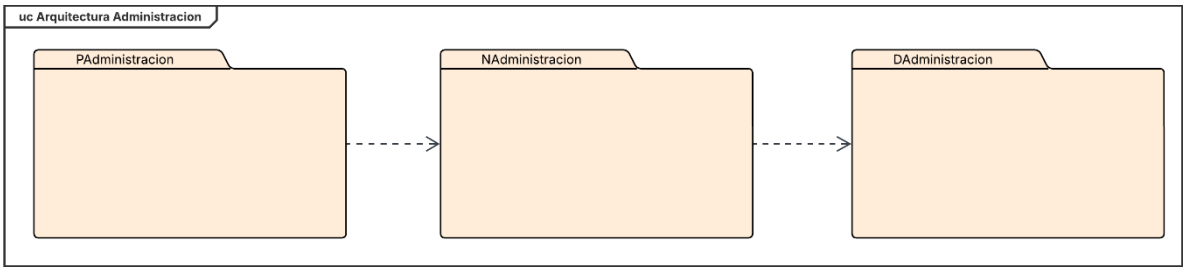
3.2.2 Modulo Asistencia



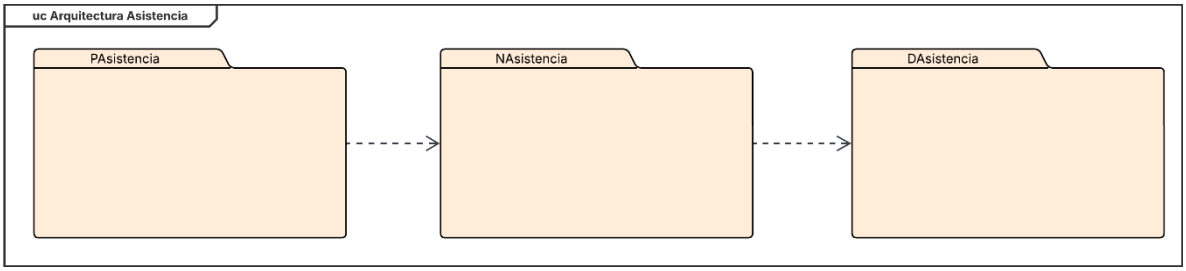
4 DISEÑO

4.1 Diseño de la Arquitectura Lógica

4.1.1 Administración

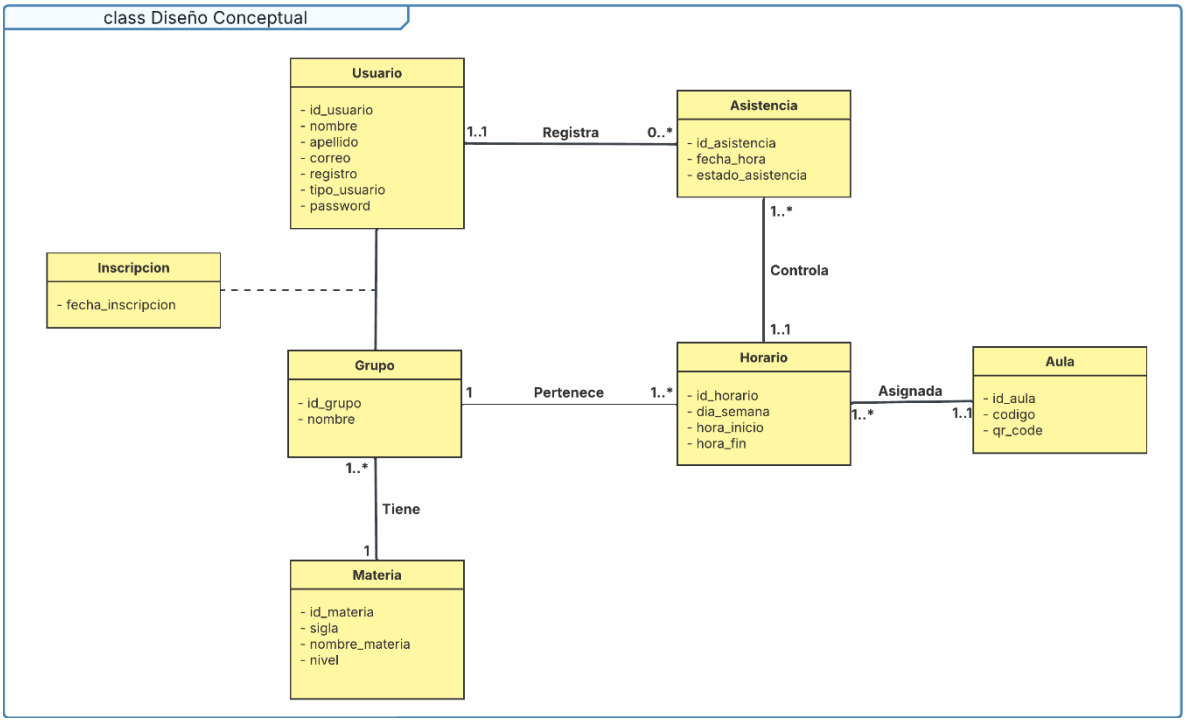


4.1.2 Asistencia



4.2 Diseño de la base de datos

4.2.1 Diseño conceptual



4.2.2 Diseño Lógico

4.2.2.1 Tabla Usuario

PK						
Id_usuario	nombre	apellido	correo	registro	tipo_usuario	password

4.2.2.2 Tabla Materia

PK			
id_materia	sigla	nombre_materia	nivel

4.2.2.3 Tabla Grupo

PK		**FK**
id_grupo	nombre	id_materia

4.2.2.4 Tabla Aula

PK		**FK**
id_aula	codigo	qr_code

4.2.2.5 Tabla horario

PK				**FK**	**FK**
id_horario	dia_semana	hora_inicio	hora_fin	**id_aula**	**id_grupo**

4.2.2.6 Tabla Inscripción

PK/FK	**PK/FK**	
id_estudiante	**id_grupo**	fecha_inscripcion

4.2.2.7 Tabla Asistencia

PK			**FK**	**FK**

id_asistencia	fecha_hora	estado_asistencia	**id_estudiante**	**id_horario**

4.2.3 Diseño Físico

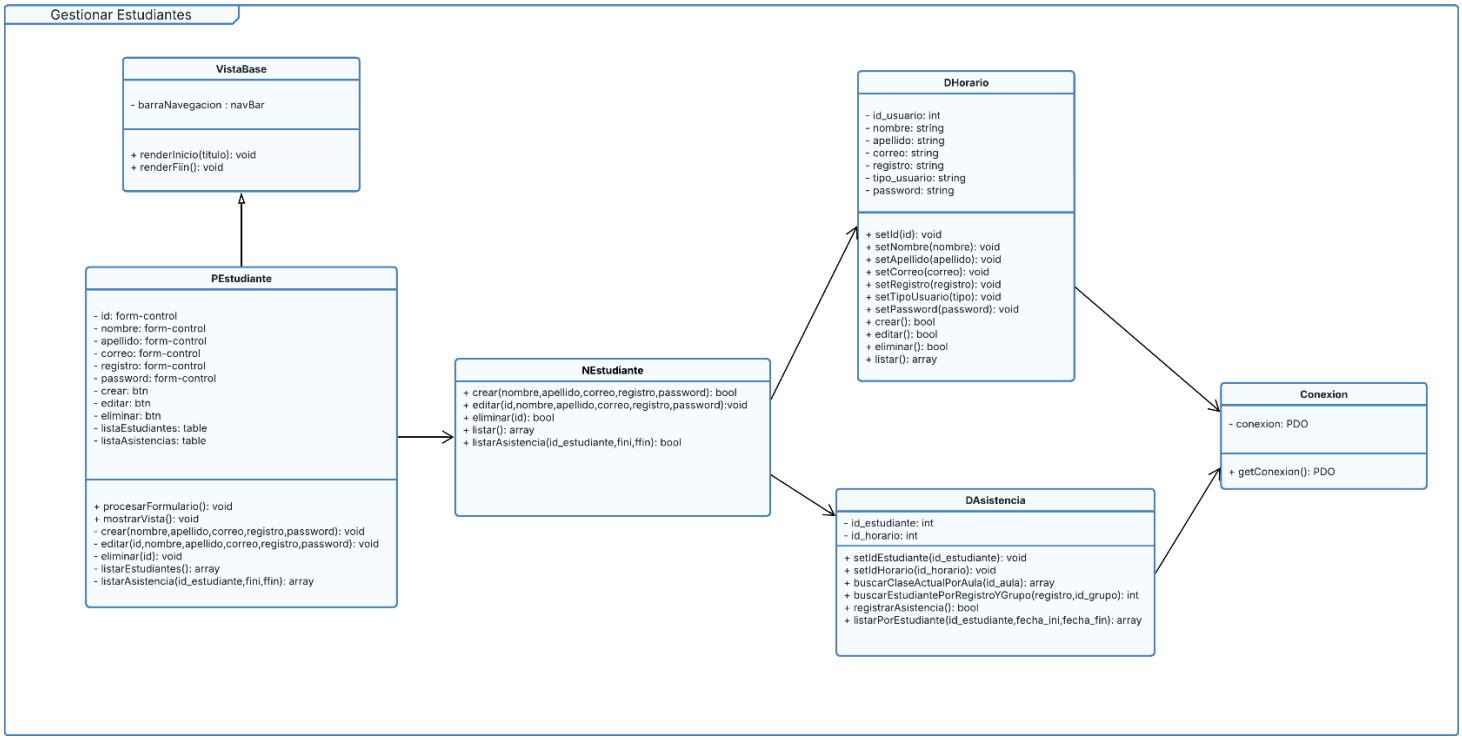
```
-- USUARIO
CREATE TABLE usuario (
id_usuario SERIAL PRIMARY KEY,
nombre VARCHAR(80) NOT NULL,
apellido VARCHAR(80) NOT NULL,
correo VARCHAR(160) NOT NULL UNIQUE,
registro VARCHAR(40),
tipo_usuario VARCHAR(20) NOT NULL,
password TEXT NOT NULL
);

-- MATERIA
CREATE TABLE materia (
id_materia SERIAL PRIMARY KEY,
sigla VARCHAR(20) NOT NULL UNIQUE,
nombre_materia VARCHAR(120) NOT NULL,
nivel VARCHAR(20) NOT NULL
);

-- GRUPO
CREATE TABLE grupo (
id_grupo SERIAL PRIMARY KEY,
id_materia INT NOT NULL,
nombre VARCHAR(80) NOT NULL,
CONSTRAINT grupo_unico_por_materia UNIQUE (id_materia, nombre),
FOREIGN KEY (id_materia) REFERENCES materia(id_materia) ON DELETE
RESTRICT
);

-- AULA
CREATE TABLE aula (
id_aula SERIAL PRIMARY KEY,
codigo VARCHAR(40) NOT NULL UNIQUE,
qr_code TEXT
);

-- HORARIO
CREATE TABLE horario (
id_horario SERIAL PRIMARY KEY,
id_aula INT NOT NULL,
id_grupo INT NOT NULL,
dia_semana SMALLINT NOT NULL,
hora_inicio TIME NOT NULL,
hora_fin TIME NOT NULL,
FOREIGN KEY (id_aula) REFERENCES aula(id_aula) ON DELETE RESTRICT,
```

```
FOREIGN KEY (id_grupo) REFERENCES grupo(id_grupo) ON DELETE CASCADE
);

-- INSCRIPCION
CREATE TABLE inscripcion (
id_estudiante INT NOT NULL,
id_grupo INT NOT NULL,
fecha_inscripcion DATE NOT NULL DEFAULT CURRENT_DATE,
CONSTRAINT inscripcion_unica UNIQUE (id_estudiante, id_grupo),
PRIMARY KEY (id_estudiante, id_grupo)
FOREIGN KEY (id_estudiante) REFERENCES usuario(id_usuario) ON DELETE
CASCADE,
FOREIGN KEY (id_grupo) REFERENCES grupo(id_grupo) ON DELETE CASCADE
);

-- ASISTENCIA
CREATE TABLE asistencia (
id_asistencia SERIAL PRIMARY KEY,
id_estudiante INT NOT NULL,
id_horario INT NOT NULL,
fecha_hora TIMESTAMP NOT NULL DEFAULT NOW(),
estado_asistencia VARCHAR(15) NOT NULL DEFAULT 'presente',
FOREIGN KEY (id_estudiante) REFERENCES usuario(id_usuario) ON DELETE
CASCADE,
FOREIGN KEY (id_horario) REFERENCES horario(id_horario) ON DELETE CASCADE
);
```

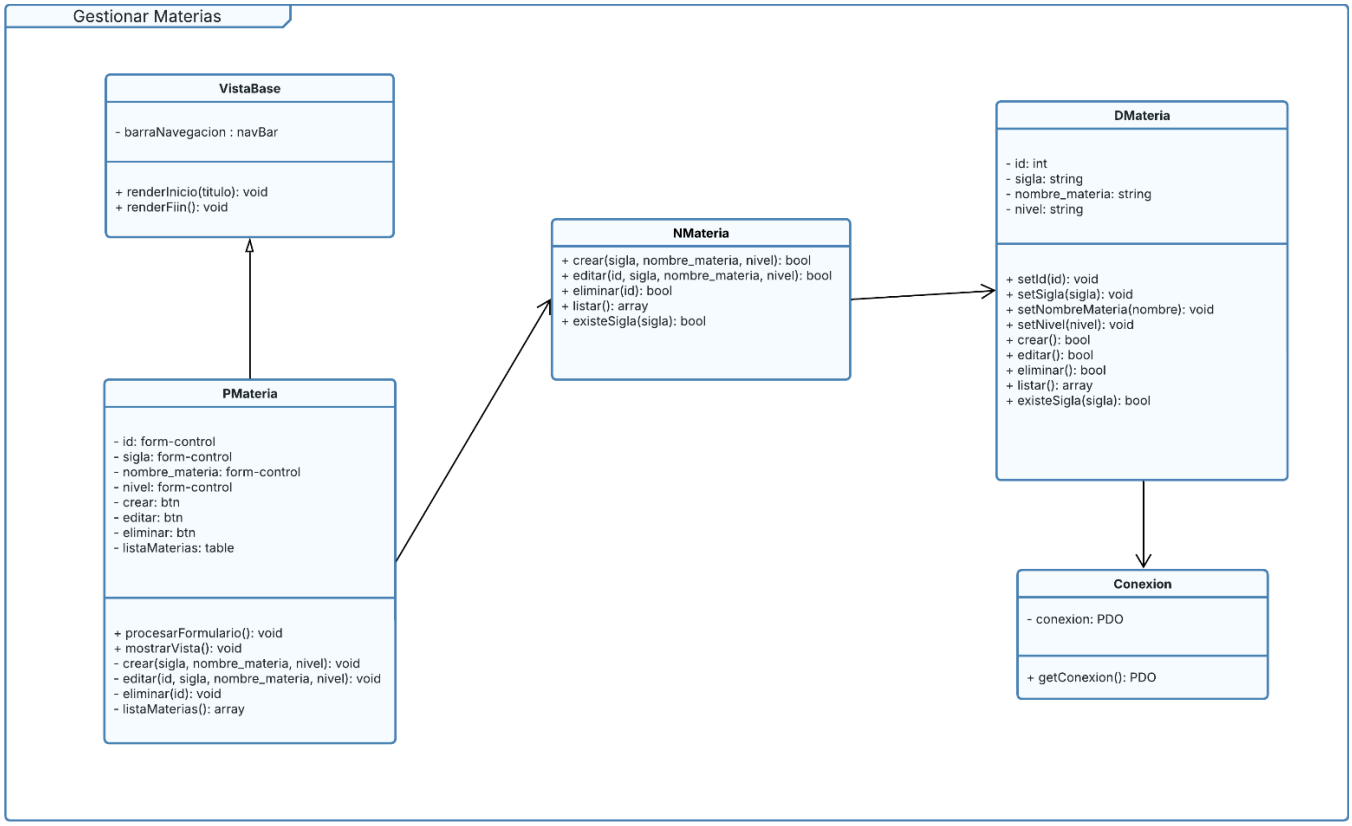
4.3 Diseño de Detalle Procedimental

4.3.1 Diagrama de clases dinamico

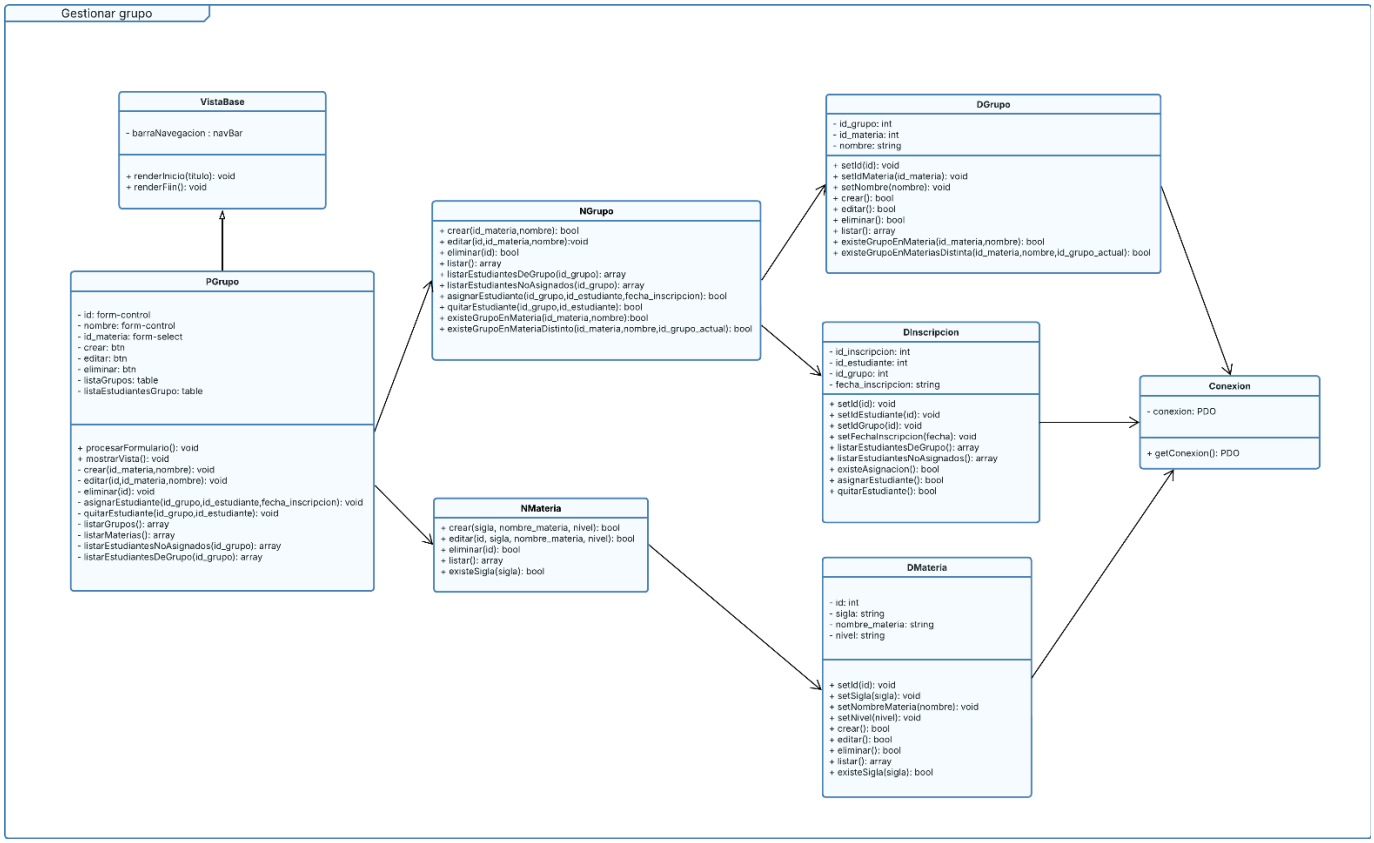
4.3.1.1 CU01: Gestionar Estudiantes



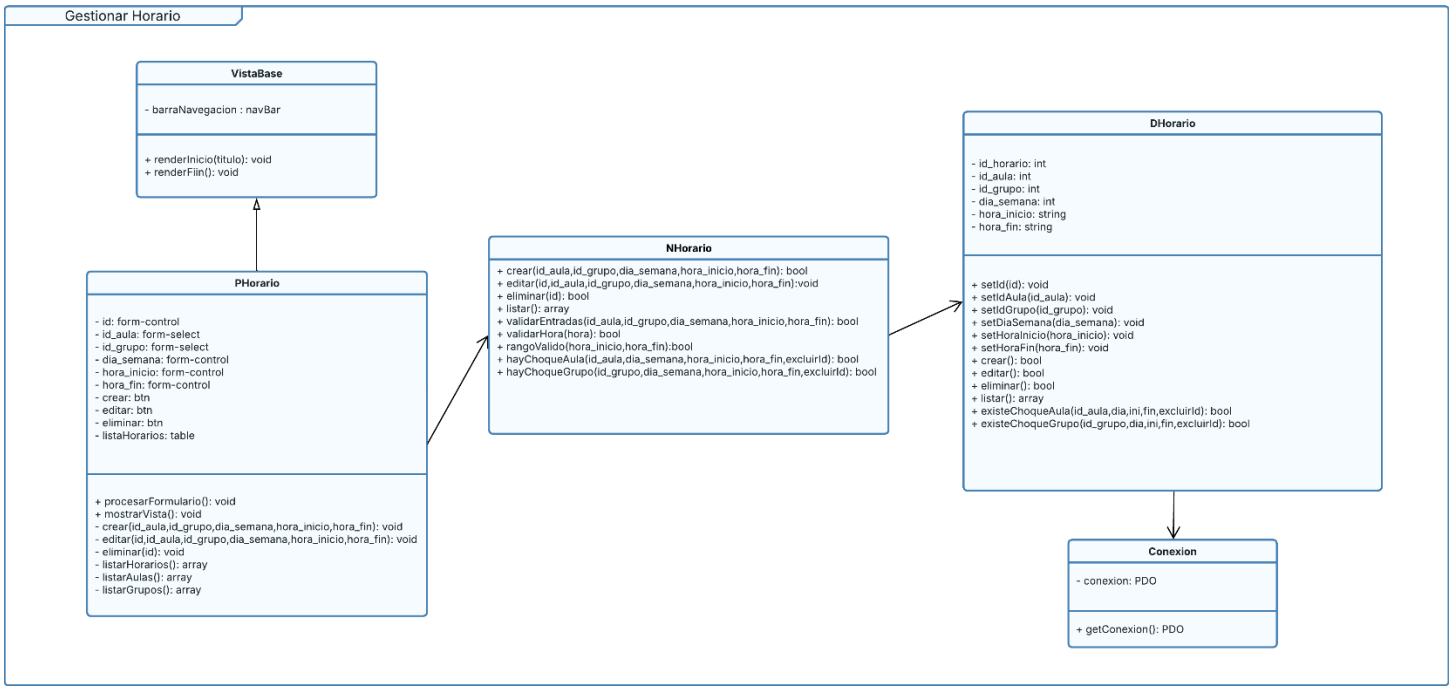
4.3.1.2 CU02: Gestionar Materias



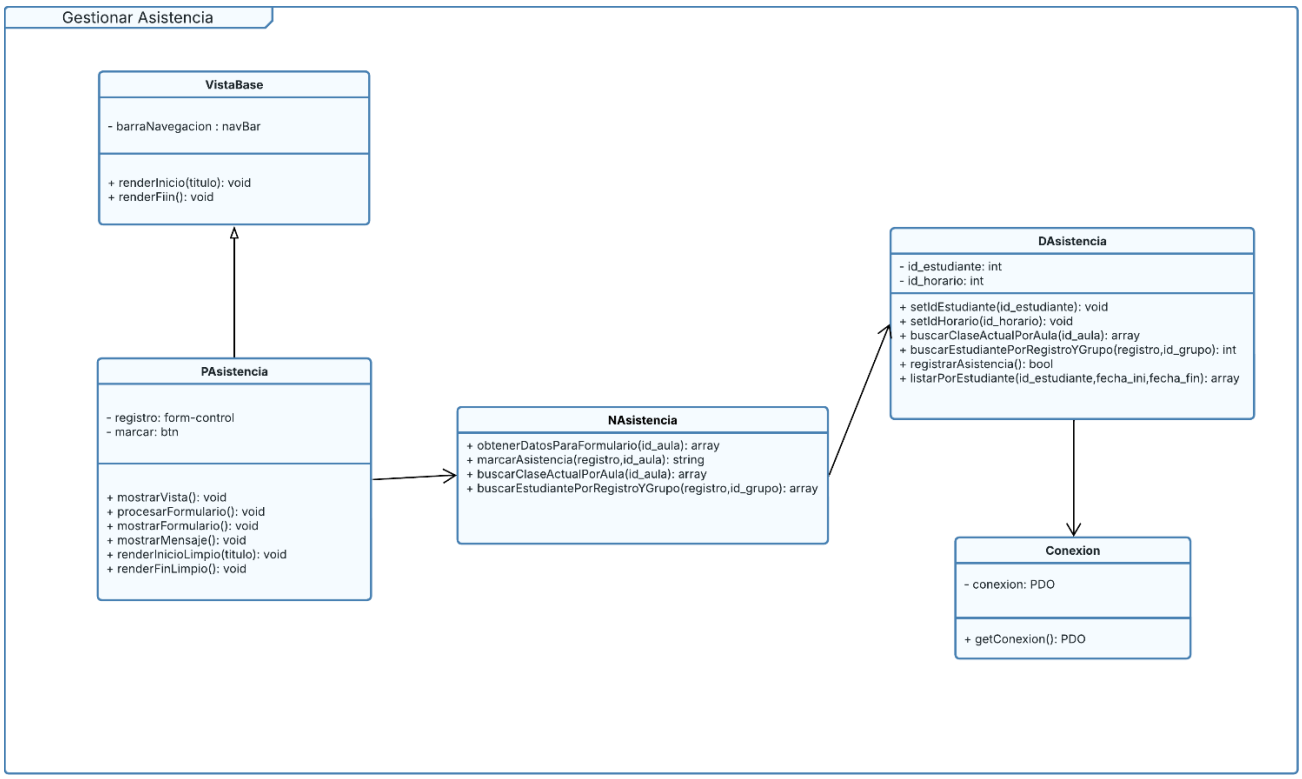
4.3.1.3 CU03: Gestionar Grupo



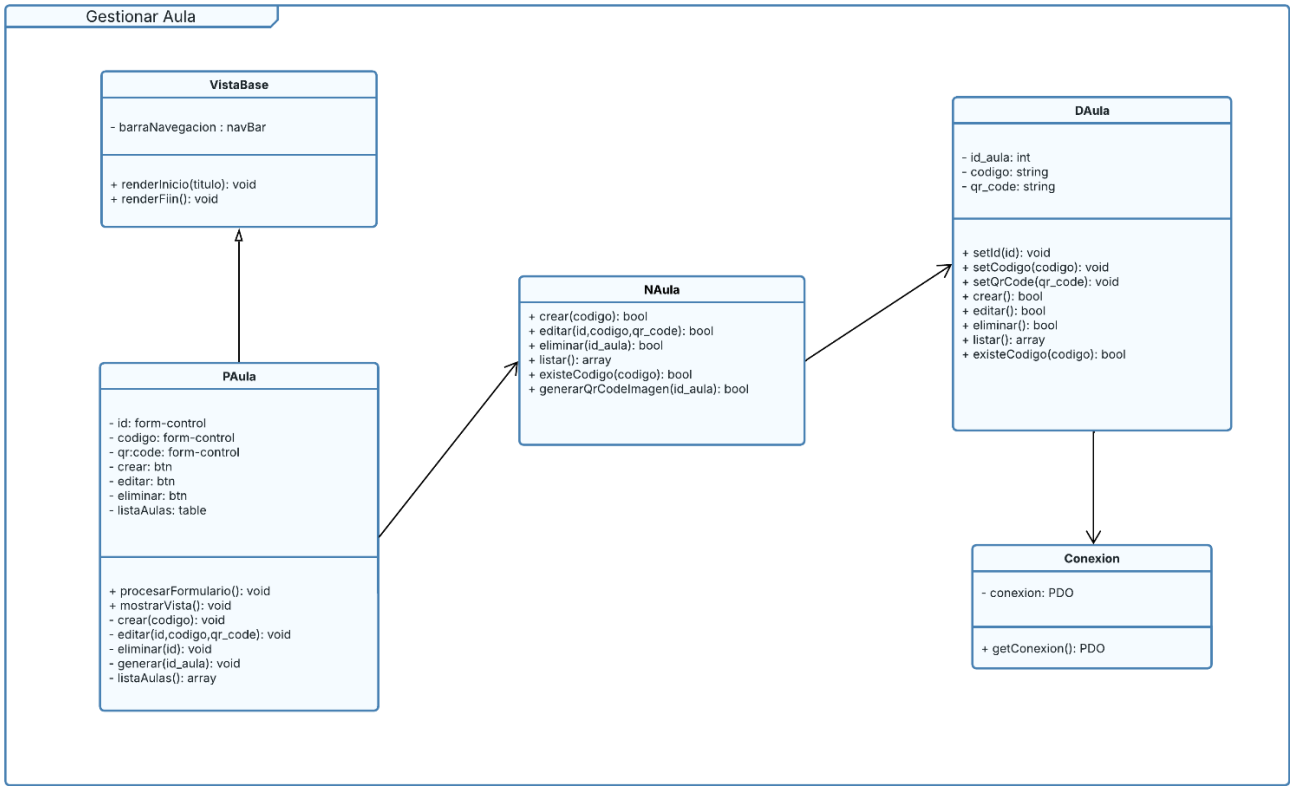
4.3.1.1 CU04: Gestionar Horario



4.3.1.1 CU05: Registrar Asistencia QR

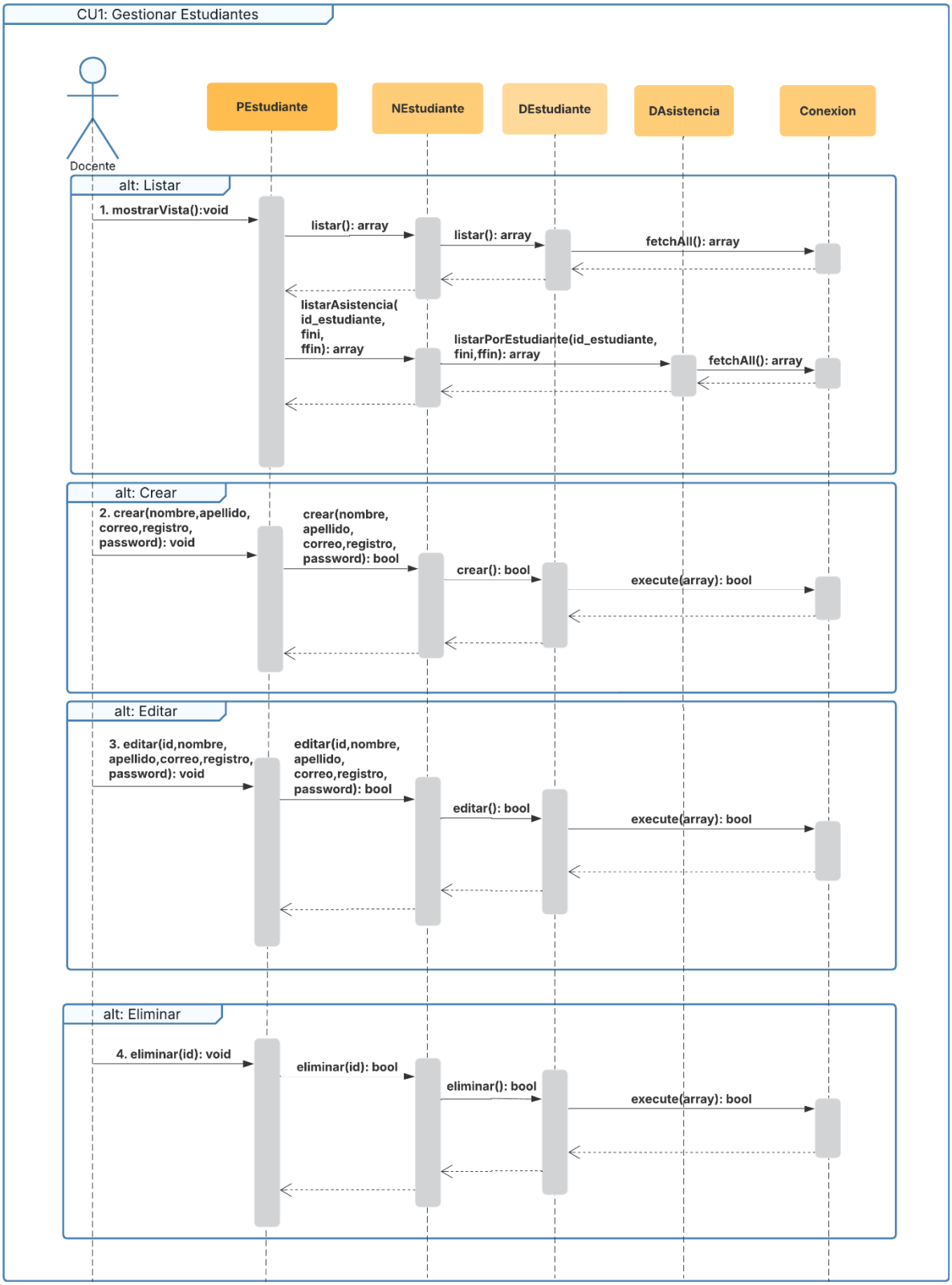


4.3.1.1 CU06: Gestionar Aula

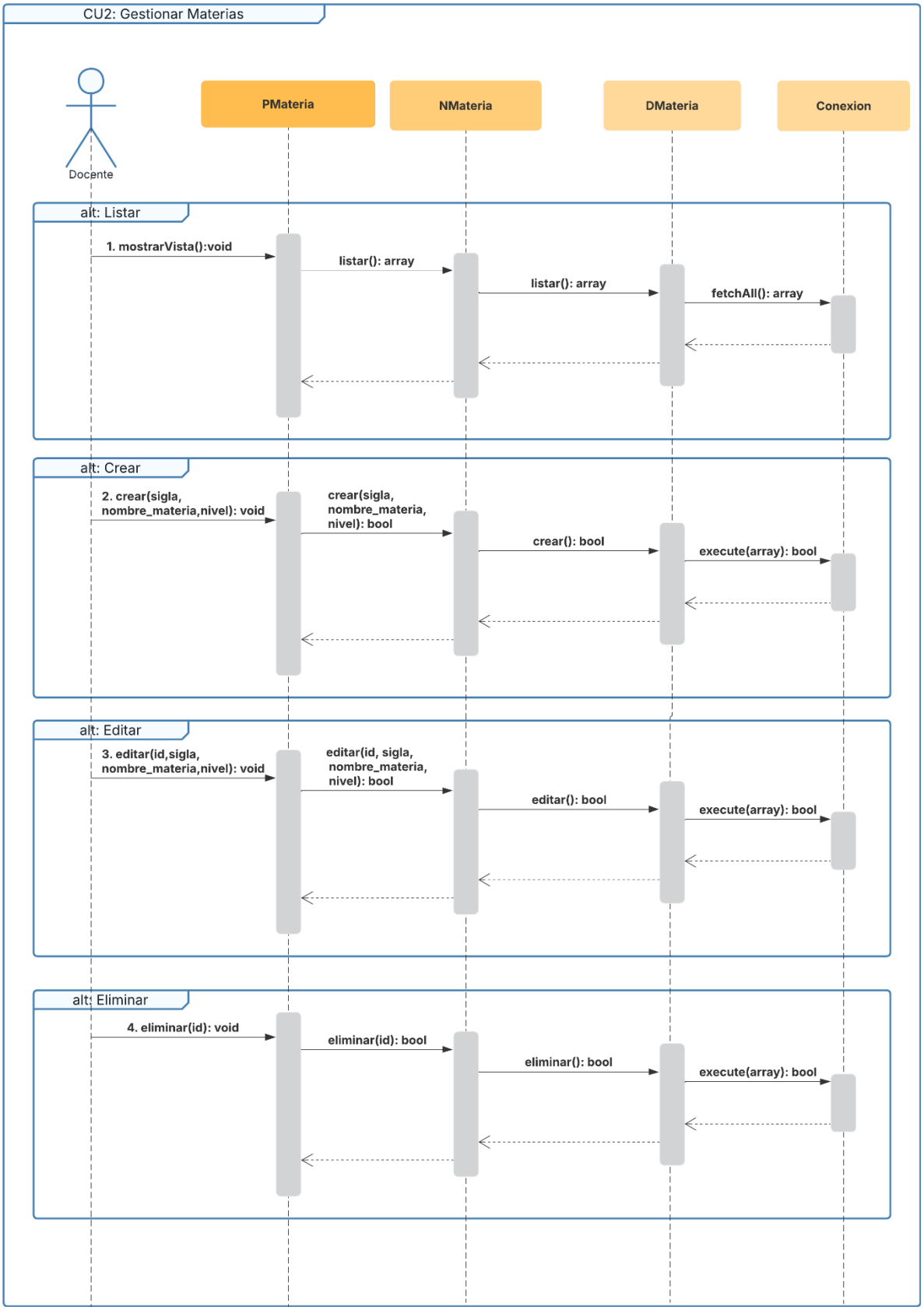


4.3.1 Diagrama de secuencia

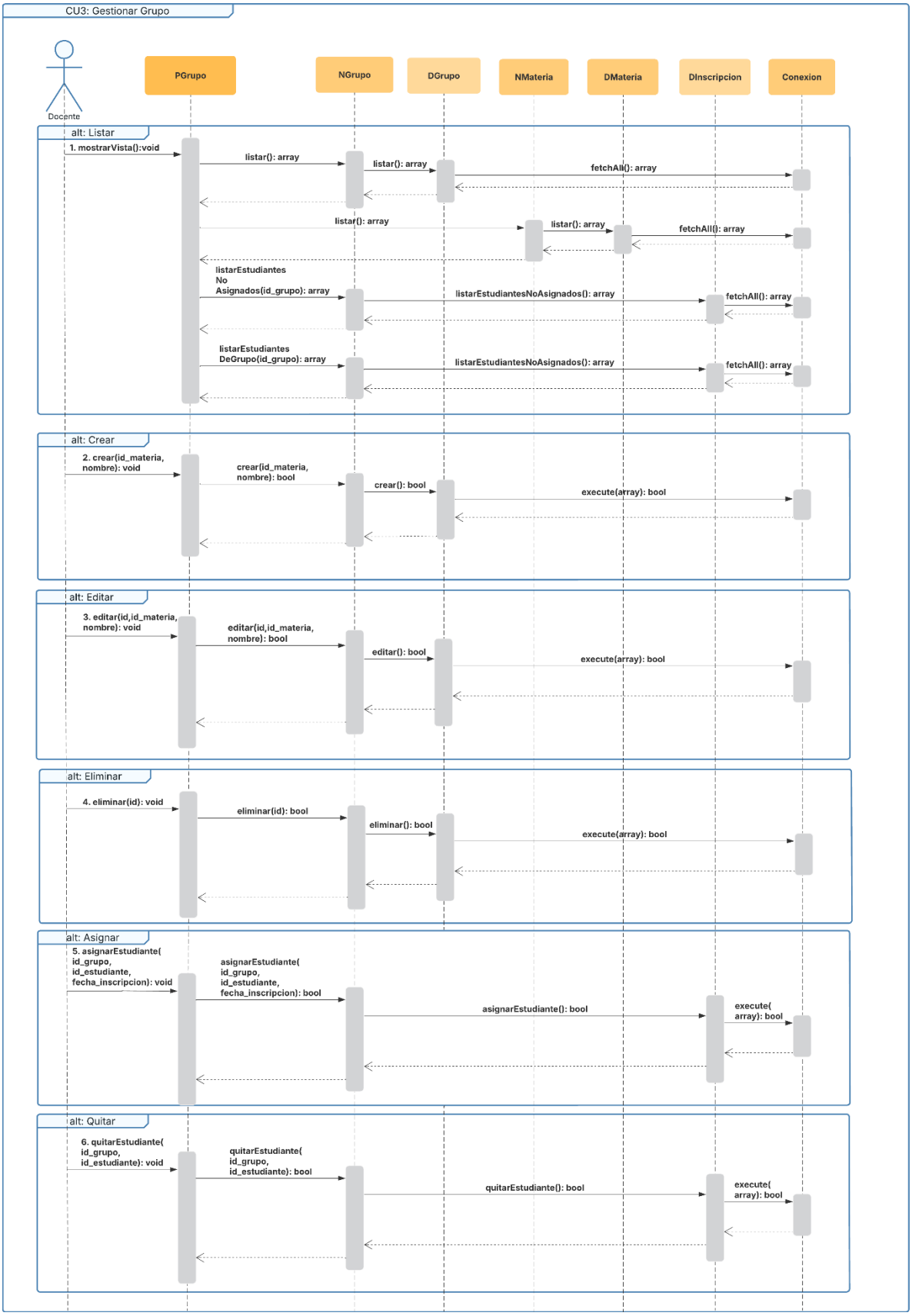
4.3.1.1 CU01: Gestionar Estudiantes



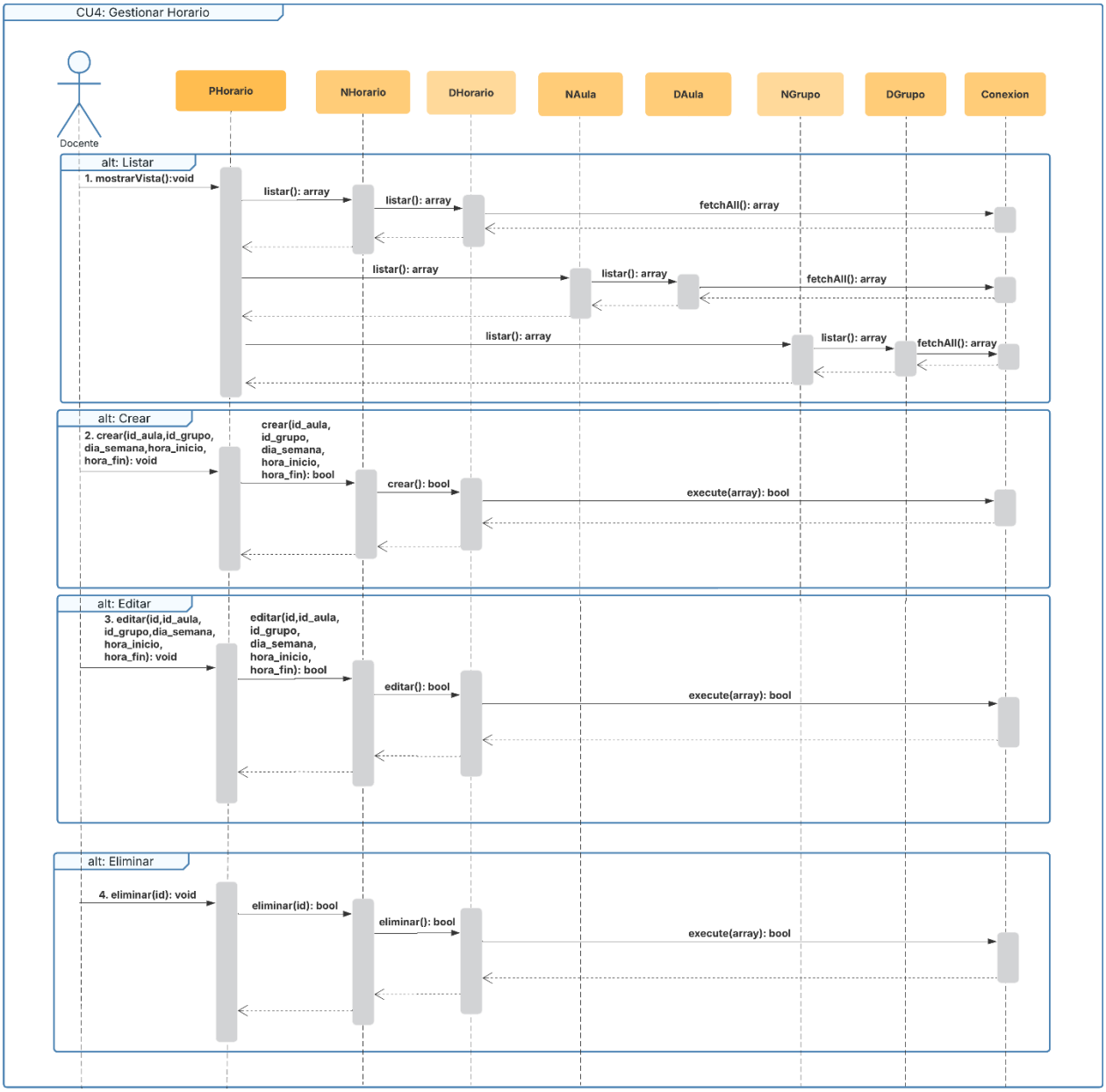
4.3.1.2 CU02: Gestionar Materias



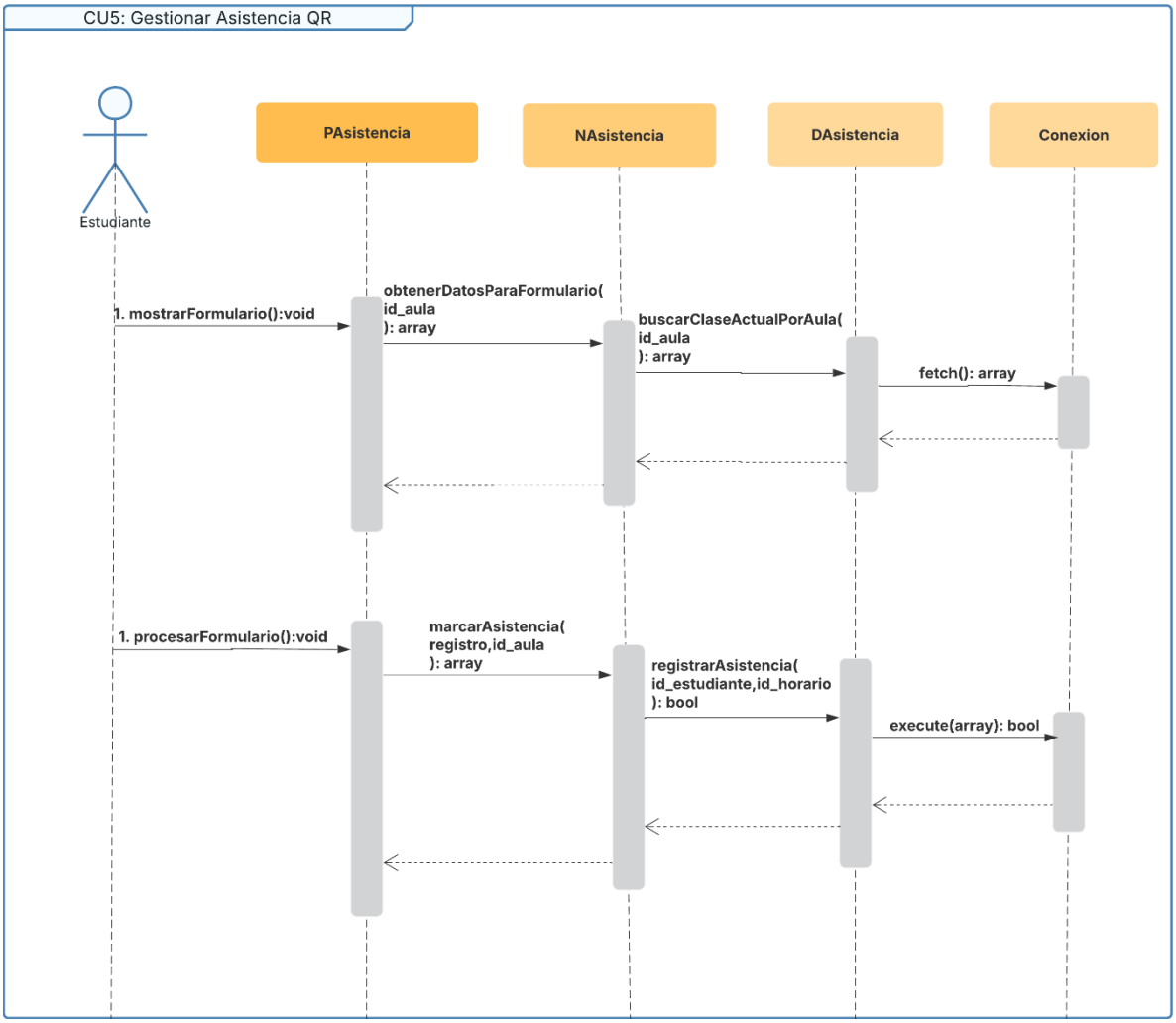
4.3.1.3 CU03: Gestionar Grupo



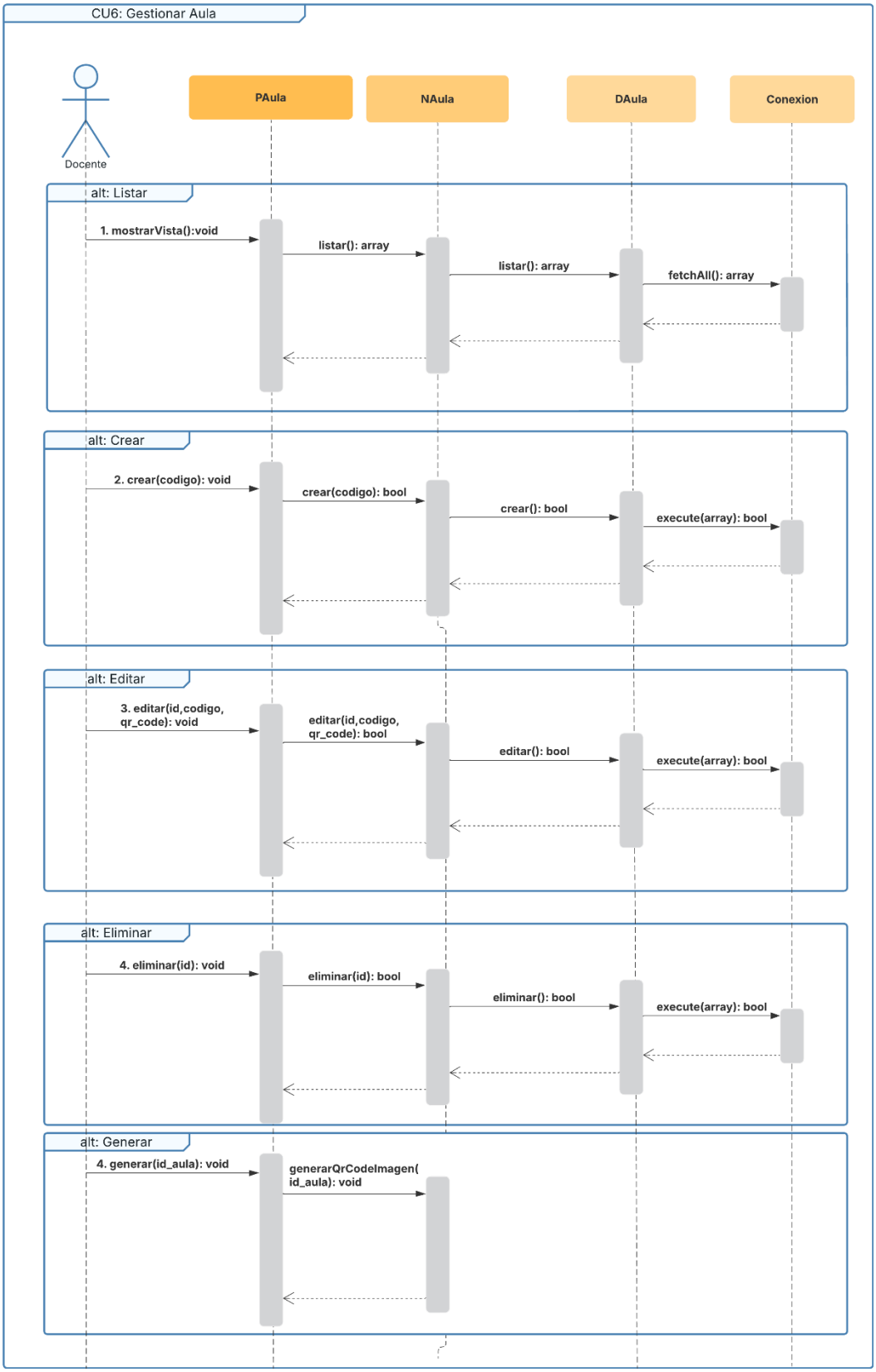
4.3.1.4 CU04: Gestionar Horario



4.3.1.5 CU05: Registrar Asistencia QR

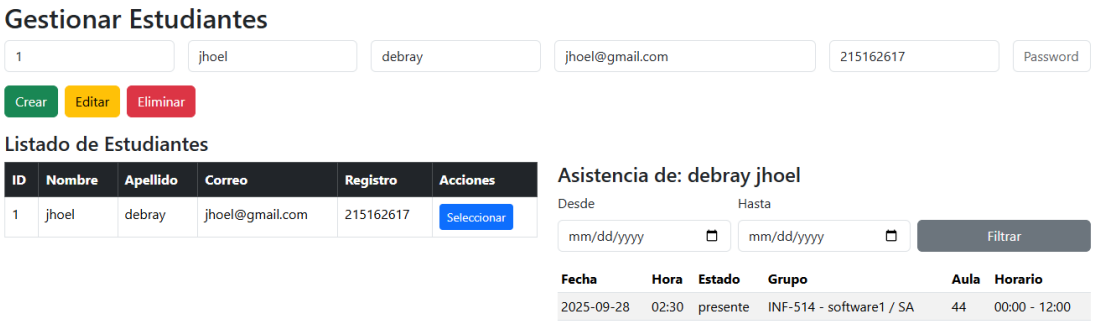


4.3.1.6 CU06: Gestionar Aula



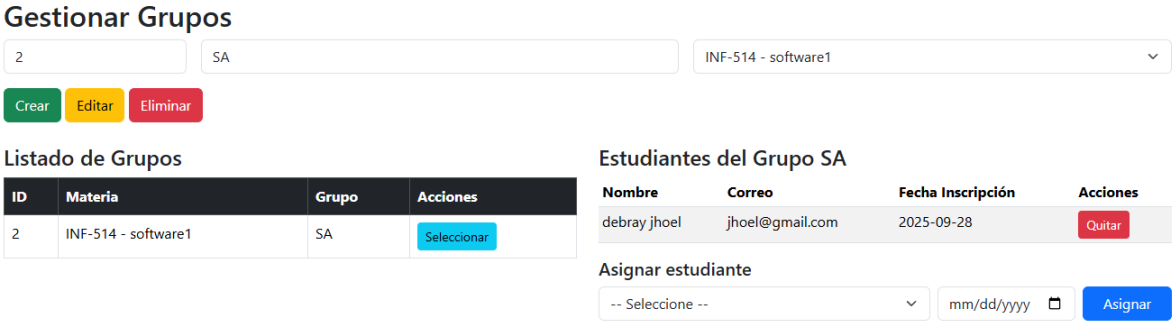
4.4 Diseño de la interfaz

4.4.1 CU01: Gestionar Estudiantes

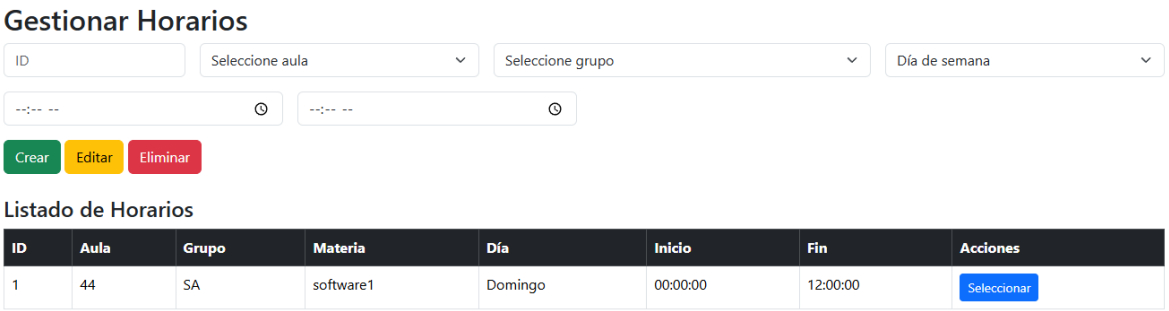


4.4.2 CU02: Gestionar Materias

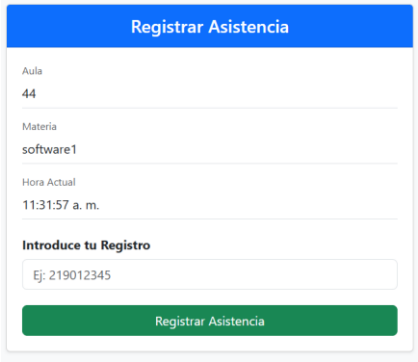
4.4.3 CU03: Gestionar Grupo



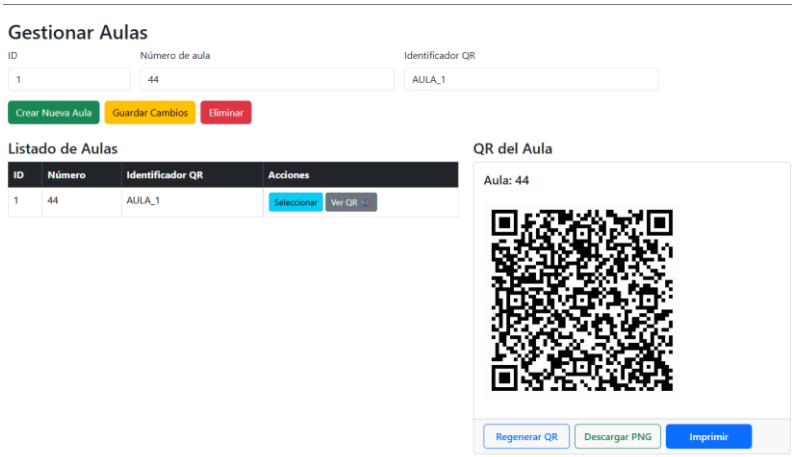
4.4.4 CU04: Gestionar Horario



4.4.5 CU05: Registrar Asistencia QR



4.4.6 CU06: Gestionar Aula



5 Bibliografia

1. JetBrains. (n.d.). *Compose Multiplatform Documentation*. Retrieved from https://www.jetbrains.com/compose/

2. Bootstrap. (n.d.). Bootstrap Documentation. Retrieved from https://getbootstrap.com/docs/

3. The PHP Group. (n.d.). PHP Manual. Retrieved from https://www.php.net/manual/en/

4. Microsoft. (n.d.). Visual Studio Code Documentation. Retrieved from https://code.visualstudio.com/docs

5. The PostgreSQL Global Development Group. (n.d.). PostgreSQL Documentation. Retrieved from https://www.postgresql.org/docs/